OOP S23 MODEL ANSWER MSBTE

This Study Material is provided by Campusifyplus.in ©

Made by Team Campusify

THIS IS THE FREE STUDY MATERIAL BY CAMPUSIFY!!

Be Part of Our Community

Telegram Channel - https://t.me/Campusifyy

Whatsapp Channel - https://tinyurl.com/Campusify-channel

Campusify is building the Biggest Diploma/Engineering Student Community in Maharashtra. Be part of the movement!

Join Channel for MSBTE Study Materials



Earn by Referring Campusify Course

Presented by **Campusify** | **10K** Active Members | **2.7L** Students Helped within **1 year**

Refer Our course with your **Third year** friend or cousin and get **10%** commission on each referral.

For More Details : WhatsApp Us at **+91 8291934640**

TY Course Link - https://www.campusifyplus.site

SUMMER – 2023 EXAMINATION
MODEL ANSWER-Only for the Use of RAC Assessors

Subject: Object Oriented Programming Using C++ **Subject Code:** **22316**

Important Instructions to examiners:

1) The answers should be examined by key words and not as word-to-word as given in the model answer scheme.
2) The model answer and the answer written by candidate may vary but the examiner may try to assess the understanding level of the candidate.
3) The language errors such as grammatical, spelling errors should not be given more Importance (Not applicable for subject English and Communication Skills.
4) While assessing figures, examiner may give credit for principal components indicated in the figure. The figures drawn by candidate and model answer may vary. The examiner may give credit for anyequivalent figure drawn.
5) Credits may be given step wise for numerical problems. In some cases, the assumed constant values may vary and there may be some difference in the candidate's answers and model answer.
6) In case of some questions credit may be given by judgement on part of examiner of relevant answer based on candidate's understanding.
7) For programming language papers, credit may be given to any other program based on equivalent concept.
8) As per the policy decision of Maharashtra State Government, teaching in English/Marathi and Bilingual (English + Marathi) medium is introduced at first year of AICTE diploma Programme from academic year 2021-2022. Hence if the students in first year (first and second semesters) write answers in Marathi or bilingual language (English +Marathi), the Examiner shall consider the same and assess the answer based on matching of concepts with model answer.

Q. No	Sub Q.N.	Answer	Marking Scheme

1.	**a) Ans.**	**Attempt any FIVE of the following:**
		Demonstrate the static and dynamic initialization of variables
		Static initialization
		Syntax-
		datatype variable_name=value;
		Example -
		int a=5;
		or
		int a;
		a=5;
		
		Dynamic initialization
		Syntax –
		int a;
			10M
			2M
			Static initialization -1M, dynamic initialization -1M

		cout<<"Enter value of variable "; cin>>variable_name; **Example-** int a; cout<<"Enter value of variable a"; cin>>a; ***Note: Syntax / Example shall be considered***	

	b) Ans.	**Write the syntax for declaration of a class**
		class class_name
		{
		private:
		variable declarations;
		function declarations;
		public:
		variable declarations;
		function declarations;
		};
			2M
			Correct syntax 2M

	c) Ans.	**State the characteristics of static member function** • A static function can have access to only other static members declared in the class. • A static member function can be called using the class name instead of objects as follows: class_name::function_name;	**2M** ***Any two characteristics 1M each***
	d) Ans.	**State different types of visibility modes in inheritance** • private • public • protected	**2M** ***Any two types 1M each***

	e) Ans.	**Give the syntax for constructor in derived classes**
		Derived_constructor (arglist1, arglist2,…, arglistN,):
		base1(arglist2), . . . ,baseN(arglistN)
		{
		Body of derived class constructor
		}
			2M
			Correct syntax 2M

	f) Ans.	**Define polymorphism with its types** Polymorphism is the ability to take more than one form.	**2M** ***Definition 1M***

		Types: 1. Compile time polymorphism 2. Run time polymorphism	***Types 1M***
	g) **Ans.**	**Define file with its operations** A file is a collection of related data stored in a particular area on the disk. **File Operations:** Open file Close file Read file Write file Update file	**2M** ***Definition 1M*** ***Any two operations 1M***
2.	**a)** **Ans.**	**Attempt any <u>THREE</u> of the following:** **State the features of object oriented programming.** Features of Object Oriented Programming are: • Follows bottom-up approach in program design. • Emphasis is on data rather than procedure. • Programs are divided into what are known as objects. • Data structures are designed such that they characterize the objects • Functions that operate on the data of an object are tied together in the data structures. • Data is hidden and cannot be accessed by external functions. • Objects may communicate with each other through functions. • New data and functions can be easily added whenever necessary	**12M** **4M** ***Any four features 1M each***
	b) **Ans.**	**Explain overloaded constructor with suitable example.** • Constructor overloading is a process of defining more than one constructor function in a class. • A class can have more than one constructor function • Type of constructor: default constructor, parameterized constructor, constructor with default value and copy constructor. • An appropriate constructor function executes when number of parameters and its data type matches with constructor function prototype.	**4M** ***Explanation 2M***

Example:

```
class example
{
int a,b;
public:
example()
{
a=0;
b=0;
}
example(int x)
{
a=x;
}
example(int x,int y)
{
a=x;
b=y;
}
};
void main()
{
example I1;
example I2(10);
example I3(10,20);
}
```

In the above code, example is the name of the class.
First constructor is a default constructor that receives no arguments. When object I1 is created without values, first constructor executes and assigns 0 value to variables a and b.
Second constructor receives one argument. When object I2 is created with one value, second constructor executes and assigns value 10 to variable a.
Third constructor receives two arguments. When object I3 is created with two values, third constructor executes and assigns value 10 to variable a and value 20 to variable b.

Example 2M

	c) Ans.	**Write a program on single inheritance**
		#include<iostream.h>
		#include<conio.h>
		class employee
		{
		protected:
		int emp_id;
		char name[10];
		};
		class emp_info:public employee
		{
		int basic_salary;
		public:
		void getdata()
		{
		cout<<"Enter emp id";
		cin>>emp_id;
		cout<<"Enter name";
		cin>>name;
		cout<<"Enter basic salary";
		cin>>basic_salary;
		}
		void putdata()
		{
		cout<<"\nEmp_id="<<emp_id;
		cout<<"\nName="<<name;
		cout<<"\nBasic Salary="<<basic_salary;
		}
		};
		void main()
		{
		emp_info e;
		clrscr();
		e.getdata();
		e.putdata();
		getch();
		}
		
		Note: Any other relevant program shall be considered.
			4M
			Correct logic 2M
			Correct syntax 2M

	d) Ans.	**Illustrate this pointer with example.** • C++ uses a unique keyword called 'this' to represent an object that invokes a member function.	**4M**

		• 'this' unique pointer is automatically passed to a member function when it is invoked.
		• 'this' is a pointer that always point to the object for which the member function is called.
		• For example, the function call A.max ( ) will set the pointer 'this' to the address of the object A. Then suppose we call B.max ( ), the pointer 'this' will store address of object B.
		
		Example:
		#include<iostream.h>
		class sample
		{
		int a;
		public:
		void setdata(int x)
		{
		this ->a=x;
		}
		void putdata()
		{
		cout<<this ->a;
		}
		};
		void main()
		{
		sample s;
		s.setdata(100);
		s.putdata( );
		}
		In the above example, this pointer is used to represent object s when setdata() and putdata ( ) functions are called.
		
		Note: Any other relevant example shall be considered.
			Explanation 2M
			
			Example 2M

3.	**a)** **Ans.**	**Attempt any THREE of the following:** **Write a program to print first n natural numbers and their sum using for loop.** #include<iostream.h> #include<conio.h>	**12M** **4M**

```
void main()
{
        int n, sum = 0;
        clrscr();
        cout<< "Enter a positive integer:";
        cin>> n;
        cout<<"\nNatural numbers till "<<n<<" numbers are:";
        for(inti = 1; i<= n; i++)
        {
                cout<<i<<" ";
        }
        cout<<"\n";
        for (i = 1; i<= n; i++)
        {
                sum=sum+i;
        }
        cout<< "Sum of numbers is = " << sum;
getch();
}
```

Declaration 1M, Display of natural numbers 1M,

Calculate and display sum 2M

b) Ans. **Compare static and non static data members (any four points)** — **4M**

Any four correct points 1M each

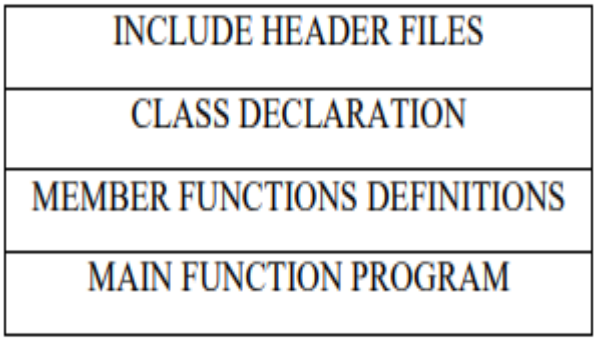
Static data member	Non static data member
1.It is declared using keyword 'static'.	1.It is declared without using keyword 'static'.
2. Static data member is automatically initialized with zero.	2. Non-static data member can be initialized with zero or non-zero value.
3.All objects of a class share the same copy of static data members.	3.Each object of the class gets its own copy of non-static data members.
4.It can be accessed through static member function as well as non-static member function of the class	4.Itcan be accessed only through non-static member function of the class
5. Declaration Syntax: static data_type variable; e.g. static int count;	5.Declaration Syntax: data_type variable; e.g. int no;

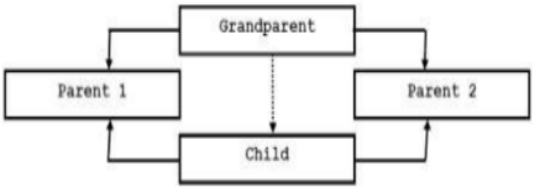
	c) Ans.	**State any four rules for virtual functions** **Rules for virtual function are:** 1. A virtual function must be a member of some class. 2. Virtual functions cannot be static members. 3. Virtual functions can be a friend of another class. 4. Virtual functions should be accessed using pointers. 5. A virtual function in based class must be defined, even though it may not be used. 6. The prototype of virtual functions should be same in base as well as in derived class. 7. If it is defined in base class, it need not be necessarily redefined in derived class. 8. A class may have virtual destructor, but it cannot have a virtual constructor. 9. While a base pointer can point to any type of derived object, the reverse is not true. **10.** When a base pointer points to a derived class, incrementing or decrementing it will not make it to point to the next object of derived class.	**4M** ***Any four correct rules 1M each***

	d) Ans.	**Write a program for get and put functions**
		Program code:
		#include <iostream.h>
		int main()
		{
		int count = 0;
		char c;
		cout<< "INPUT TEXT\n";
		cin.get(c);
		while (c != '\n')
		{
		cout.put(c);
		count++;
		cin.get(c);
		}
		cout<< "\nNumber of characters = " << count << "\n";
		return 0;
		}
		Note: Any other relevant program shall be considered.
			4M
			Correct syntax of get(): 2M,
			Correct syntax of put(): 2M

SUMMER – 2023 EXAMINATION
MODEL ANSWER-Only for the Use of RAC Assessors

Subject: Object Oriented Programming Using C++ **Subject Code:** **22316**

4.	**a) Ans.**	**Attempt any THREE of the following:** **Explain the structure of C++ program** General C++ program has following structure:  **Description: -** **1. Include header files** In this section a programmer includes all header files which are required to execute a given program. The most important file is iostream.h header file. This file defines most of the C++statements like cout and cin. Without this file one cannot load the C++ program. **2. Class Declaration** In this section a programmer declares all classes which are necessary for the given program. The programmer uses general syntax of creating class. **3. Member Functions Definition** This section allows programmers to design member functions of a class. The programmer can have an inside declaration of a function or outside declaration of a function. **4. Main Function Program** In this section programmers create objects and call various functions writer within various class.	**12M** **4M** ***Correct diagram 2M*** ***Correct explanation 2M***

	b) Ans.	**List any four properties of constructor function** **Properties of constructor function are:** 1. A constructor name must be same as that of its class name. 2. Constructors are called automatically when the objects are created. 3. Constructors should be declared in the public section to be available to all the functions. 4. Constructors do not have return type, not even void and therefore they cannot return value. 5. Constructors can have default arguments as other C++ functions. 6. Constructors cannot be inherited. 7. Constructors cannot be static. 8. Constructors cannot be virtual. 9. The address of a constructor cannot be referred. **10.** Constructors are member functions so they can be overloaded.	**4M** ***Any four properties 1M each***
	c) Ans.	**Illustrate the concept of virtual base class with suitable example.** Virtual Base Class: An ancestor class is declared as virtual base class to avoid duplication of inherited members inside child class due to multiple paths of inheritance.  Consider a hybrid inheritance as shown in the above diagram. The child class has two direct base classes, Parent1 and Parent2 which themselves have a common base class as Grandparent. The child inherits the members of Grandparent via two separate paths. All the public and protected members of Grandparent are inherited into Child twice, first via Parent1 and again via Parent2. This leads to duplicate sets of the inherited members of Grandparent inside Child class. The duplication of inherited members can be avoided by making the common base class as virtual base class while declaring the direct or intermediate base classes as shown below.	**4M** ***Description 2M*** ***Example 2M***

```
class Grandparent
{
};
class Parent1:virtual public Grandparent
{
};
class Parent2:virtual public Grandparent
{
};
class Child: public Parent1,public Parent2
{
};
```

Example:

```
#include<iostream.h>
#include<conio.h>
class student
{
intrno;
public:
void getnumber()
{
cout<<"Enter Roll No:";
cin>>rno;
}
void putnumber()
{
cout<<"\n\n\t Roll No:"<<rno<<"\n";
}
};
class test: virtual public student
{
public:
int part1,part2;
void getmarks()
{
cout<<"Enter Marks\n";
cout<<"Part1:";
cin>>part1; cout<<"Part2:";
```

```
cin>>part2;
}
void putmarks()
{
cout<<"\t Marks Obtained\n";
cout<<"\n\t Part1:"<<part1;
cout<<"\n\tPart2:"<<part2;
}
};
class sports: public virtual student
{
public:
int score;
void getscore()
{
cout<<"Enter Sports Score:";
cin>>score;
}
void putscore()
{
cout<<"\n\t Sports Score is:"<<score;
}
};
class result: public test, public sports
{
int total;
public:
void display()
{
total=part1+part2+score;
putnumber();
putmarks();
putscore();
cout<<"\n\t Total Score:"<<total;
}
};
void main()
{
result obj;
```

		clrscr();
		obj.getnumber();
		obj.getmarks();
		obj.getscore();
		obj.display();
		getch();
		}	

	d) Ans.	**State the advantages of pointer** 1. Pointers save memory space. 2. Pointers reduce the length and complexity of a program. 3. Pointers allow the passing of arrays and strings to function more efficiently. 4. Pointers make it possible to return more than one value from the function. 5. Execution time with pointers is faster because data are manipulated with the address, that is, direct access to memory location. 6. Memory is accessed efficiently with the pointers. The pointer assigns and releases the memory as well. Hence it can be said the Memory of pointers is dynamically allocated. 7. Pointers are used with data structures. They are useful for representing two-dimensional and multi-dimensional arrays. 8. An array, of any type, can be accessed with the help of pointers, without considering its subscript range. 9. Pointers are used for file handling. 10. Pointers are used to allocate memory dynamically. 11. In C++, a pointer declared to a base class could access the object of a derived class. However, a pointer to a derived class cannot access the object of a base class.	**4M** ***Any four correct advantages 1M each***
	e) Ans.	**Write a program for closing a file** **Program code:** #include <iostream> #include <fstream> int main() {	**4M**

		fstreamFileName;
		FileName.open("FileName.txt", ios::in);
		if (!FileName)
		{
		cout<<"File doesn't exist";
		}
		else
		{
		cout<<"File opened successfully";
		}
		FileName.close();
		return 0;
		}
			File object creation 1M, opening of file 1M, closing of file 2M

5.	**a)**
	
	Ans.
		Attempt any <u>TWO</u> of the following:
		State the use of scope resolution operator and explain it with example.
		• Scope of a variable extends from the point of declaration to the end of the block.
		• A variable declared inside a block is 'local' variable and a variable declared outside block is called as global variable.
		• When we want to use a global variable but also has a local variable with same name.
		• To access the global version of the variable, C++ provides scope resolution operator.
		
		Example:-
		#include <iostream>
		using namespace std;
		char a = 'm';
		static int b = 50;
		int main() {
		char a = 's';
		cout<< "The value static variable b is : "<< ::b;
		cout<< "\nThe value of local variable a is : " << a;
		cout<< "\nThe value of global variable a is : " << ::a;
		return 0;
		}
		Note: Any other relevant example shall be considered.
			12M
			6M
			
			Explanation 3M
			
			Example 3M

	b) Ans.	**Write a program to show object as function argument.**
		#include <iostream>
		class Decimal
		{
		private:
		int a;
		public:
		void set(int x)
		{
		a = x;
		}
		void sum(Decimal ob1, Decimal ob2)
		{
		a = ob1.a + ob2.a;
		}
		void print()
		{
		cout<< "Value of A: " << a <<endl;
		}
		};
		int main()
		{
		Decimal d1;
		Decimal d2;
		Decimal d3;
		d1.set(10);
		d2.set(20);
		d3.sum(d1, d2);
		cout<<"First Decimal value\n";
		d1.print();
		cout<<"Second Decimal value\n";
		d2.print();
		cout<<"Sum of Decimal object1 and object2 =\n";
		d3.print();
		return 0;
		}
		
		Note: Any other relevant program shall be considered.
			6M
			Correct logic 3M
			Correct syntax 3M

	c) Ans.	**Write a program on hybrid inheritance**
		#include <iostream>
		class A
		{
		public:
		int x;
		};
		class B : public A
		{
		public:
		B() //constructor to initialize x in base class A
		{
		x = 10;
		}
		};
		class C
		{
		public:
		int y;
		C() //constructor to initialize y
		{
		y = 4;
		}
		};
		class D : public B, public C //D is derived from class B and class C
		{
		public:
		void sum()
		{
		cout<< "Sum= " << x + y;
		}
		};
		int main()
			6M
			Correct logic 3M
			Correct syntax 3M

```
{
    D obj1;        //object of derived class D
  obj1.sum();
  return 0;
}
```

Note: Any other relevant program shall be considered.

6. **Attempt any TWO of the following** **12M**

a) **State the difference between constructor and destructor (any six points)** **6M**

Ans. ***Any six points 1M each***

Constructor	Destructor
1. A constructor is a special member function whose task is to initialize the objects of its class.	1. A destructor is a special s member function whose task is to destroy the objects that have been created by constructor.
2. Constructors are classified in various types such as: Overloaded constructor Default constructor Parameterized constructor Copy constructor	2. Destructors are not classified in any types.
3. It is invoked automatically when the objects are created.	3. It is invoked implicitly by the compiler upon exit of program / block / function.
4. Constructor accepts parameters. Also it can have default value for its parameter.	4. Destructor never accepts any parameter.
5. A class can have more than one constructor.	5. A class can have at the most one constructor.
6. It constructs the values of data members of the class.	6. It does not construct the values for the data members of the class.
7. Syntax- classname()	7. Syntax- destructor name is

		{… … }	preceded with tilde. ~classname() {…. …. }

		8. Example-
		ABC()
		{
		…
		}
			8. Example-
			~ABC()
			{
			….
			}

	b) Ans.	**Explain abstract class with suitable example.**
		• By definition, a C++ abstract class must include at least one pure virtual function.
		• Alternatively, put a function without a definition.
		• Because the subclass would otherwise turn into an abstract class in and of itself, the abstract class's descendants must specify the pure virtual function.
		• Although the Abstract class type cannot be created from scratch, it can have pointers and references made to it.
		• A pure virtual function can exist in an abstract class in addition to regular functions and variables.
		• The main objective of abstract base class is to provide some traits to the derived classes and to create a base pointer required for achieving run time polymorphism
		• **Example:**
		#include<iostream>
		class Figure
		{
		public:
		virtual int Area() = 0;
		void setBreadth(int br)
		{
		breadth = br;
		}
		void setHeight(int ht)
			6M
			Correct explanation 3M
			Correct example 3M

```
  {
    height = ht;
  }
protected:
    int breadth;
    int height;
 };
class Rectangle: public Figure
{
public:
int Area()
{
return (breadth * height);
 }
 };

class Triangle: public Figure
{
public:
   int Area()
  {
     return (breadth * height)/2;
  }
 };
int main()
{
 Rectangle R1;
 Triangle T1;
 R1.setBreadth(6);
 R1.setHeight(12);
 T1.setBreadth(40);
 T1.setHeight(4);
 cout<< "The area of the rectangle is: " << R1.Area() <<endl;
 cout<< "The area of the triangle is: " << T1.Area() <<endl;
}
```

Note: Any other relevant example shall be considered.

SUMMER – 2023 EXAMINATION
MODEL ANSWER-Only for the Use of RAC Assessors

Subject: Object Oriented Programming Using C++ **Subject Code: 22316**

	c)	**Write C++ program to overload binary operator '+' to concatenate two strings.**	**6M**

	Ans.	#include<iostream>
		#include<string.h>
		class String
		{
		public:
		char str[20];
		public:
		void accept_string()
		{
		cout<<"\n Enter String : ";
		cin>>str;
		}
		void display_string()
		{
		cout<<str;
		}
		String operator+(String x) //Concatenating String
		{
		String s;
		strcat(str,x.str);
		strcpy(s.str,str);
		return s;
		}
		};
		int main()
		{
		String str1, str2, str3;
		str1.accept_string();
		str2.accept_string();
		str3=str1+str2;
		cout<<"\n\n Concatenated String is : ";
		str3.display_string();
		return 0;
		}
			Correct logic 3M
			
			Correct syntax 3M